DOCUMENTAZIONE FUNZIONI

Progetto di Sistemi Operativi

Applicazione Client-Server

File: `client.c` | `server.c` | `common.h`

Linguaggio: C · POSIX Threads · Socket TCP

1. Panoramica del Progetto

Il progetto realizza un servizio di messaggistica testuale client-server in linguaggio C (C17) su sistemi POSIX-compatibili. Il server è concorrente: ogni client connesso viene servito da un thread POSIX dedicato. Client e server comunicano tramite pacchetti binari di tipo fisso (struct Packet). La persistenza è garantita da file di testo sul filesystem locale del server.

Rispetto alla versione iniziale, il codice ha ricevuto diverse evoluzioni significative: correzione del bug di scansione parziale del file utenti in user_exists() e register_user() che impediva il riconoscimento degli utenti non in prima posizione; introduzione della validazione sistematica dei caratteri proibiti ('|' e '\') lato client tramite contains_forbidden() e fgets_validated(); randomizzazione dell'ID messaggio lato server per ridurre le collisioni; meccanismo di retry della registrazione senza chiudere la connessione TCP; e unificazione dei mutex per messages.txt per eliminare la race condition strutturale tra lettura e cancellazione.

2. Struttura dei File

File	Ruolo	Descrizione
common.h	Header condiviso	Costanti, enum PacketType, struct Packet. Incluso da entrambi i sorgenti.
server.c	Processo server	Gestione connessioni concorrenti, autenticazione, archiviazione e cancellazione messaggi.
client.c	Processo client	Interfaccia utente a menu, validazione input, comunicazione con il server.
users.txt	Persistenza utenti	Credenziali nel formato 'username password\n', una coppia per riga.
messages.txt	Persistenza messaggi	Archivio messaggi in formato delimitato da pipe.
messages_tmp.txt	File temporaneo	Usato esclusivamente durante la cancellazione atomica dei messaggi.

3. Header Condiviso — common.h

3.1 Costanti

Costante	Valore	Significato
SERVER_PORT	9090	Porta TCP di ascolto del server.
MAX_CLIENTS	50	Backlog della listen(). Non limita il numero effettivo di thread concorrenti.
MAX_LEN	1024	Buffer di lettura riga per il parsing dei file di persistenza (fgets).
MAX_USER	64	Lunghezza massima (incluso '\0') di username e receiver.
MAX_PASS	64	Lunghezza massima della password.
MAX_OBJ	128	Lunghezza massima del campo subject.
MAX_BODY	1024	Lunghezza massima del corpo del messaggio.
PRINT_BUFLEN	64	Buffer per la formattazione timestamp in print_packet().
LOG_BUF	64	Buffer per il log timestamp lato server.
USERS_FILE	"users.txt"	File di persistenza delle credenziali utente.
MSG_FILE	"messages.txt"	File di persistenza dei messaggi archiviati.
TMP_FILE	"messages_tmp.txt"	File temporaneo per la cancellazione atomica.

3.2 Enumerazione PacketType

Valore	Direzione	Semantica
AUTH_REQUEST	C → S	Richiesta login o registrazione. subject='LOGIN' | 'REGISTER'.
AUTH_SUCCESS	S → C	Autenticazione o registrazione avvenuta con successo.
AUTH_FAIL	S → C	Credenziali errate (login) oppure username già presente (registrazione, con possibilità di retry).
SEND_MESSAGE	C → S	Invio di un nuovo messaggio con tutti i campi compilati e validati.
READ_MESSAGES	C → S	Richiesta della lista dei messaggi in arrivo per l'utente autenticato.
CHECK_USER	S → C	Risposta alla SEND_MESSAGE: subject='AUTH_OK' | 'NO_RECEIVER'.
DELETE_MESSAGE	C ↔ S	Richiesta cancellazione (C→S, id=target) e risposta esito (S→C): subject='OK'|'NOT_FOUND'|'ERR_IO'.

MESSAGE_LIST	S → C	Un pacchetto per ogni messaggio inviato al client durante la lettura.
END_SESSION	C → S	Chiusura pulita della sessione. Il server chiude il socket e termina il thread.

3.3 Struttura Packet

La struct Packet è l'unità atomica di trasferimento. Viene inviata e ricevuta come blocco binario di dimensione fissa sizeof(Packet), senza framing aggiuntivo. Diversi campi vengono riutilizzati con semantiche diverse a seconda del tipo di pacchetto, una scelta che riduce il numero di tipi necessari a scapito di chiarezza semantica.

Campo	Tipo	Utilizzo e riuso semantico
type	PacketType	Discriminante obbligatorio, primo campo letto dal dispatcher lato server.
sender	char[64]	Mittente del messaggio; contiene lo username in AUTH_REQUEST.
receiver	char[64]	Destinatario del messaggio.
subject	char[128]	Oggetto del messaggio; riutilizzato come flag di controllo: 'LOGIN', 'REGISTER', 'AUTH_OK', 'NO_RECEIVER', 'OK', 'NOT_FOUND', 'ERR_IO'.
body	char[1024]	Corpo del messaggio; contiene la password in AUTH_REQUEST.
timestamp	time_t	Timestamp Unix. Generato dal client, sovrascritto definitivamente dal server in handle_client() prima di chiamare save_message().
id	int	ID generato dal client con generate_message_id() e modificato lato server tramite XOR casuale prima della scrittura su file.

4. Documentazione — server.c

4.1 Variabili Globali e Strutture

Variabile / Struttura	Tipo	Scopo
file_mutex	pthread_mutex_t	Mutex unificato per messages.txt: acquisito da save_message(), read_messages() e delete_message(). Garantisce mutua esclusione completa tra tutte le operazioni sul file messaggi, eliminando la race condition strutturale della versione precedente.
register_mutex	pthread_mutex_t	Protegge users.txt in lettura (verifica unicità) e in scrittura (aggiunta utente). Acquisito da register_user() e user_exists().
auth_mutex	pthread_mutex_t	Protegge la lettura di users.txt durante il login. Separato da register_mutex per permettere parallelismo tra login e registrazione.
server_running	volatile sig_atomic_t	Flag booleano impostato a 0 da sigint_handler(). Controlla il loop di accept() nel main.
server_sock	int globale	Descrittore del socket server, globale per permetterne la chiusura dal signal handler SIGINT.
client_info	typedef struct	Struttura contenente sock (int) e addr (sockaddr_in). Allocata con malloc() nel main per ogni nuova connessione e passata al thread worker via void*.

4.2 Funzioni

sigint_handler()

`sigint_handler` [server.c]	
Firma	`void sigint_handler(int signo)`
Descrizione	Handler POSIX per SIGINT (Ctrl+C). Imposta server_running a 0 e chiude server_sock, causando il ritorno di accept() con errore e permettendo al loop principale di uscire in modo pulito.
Parametri	`signo` — Numero del segnale. Cast a (void) per sopprimere il warning di parametro inutilizzato.
Valore di ritorno	void.

register_user()

register_user [server.c]	
Firma	`static int register_user(const char *user, const char *pass)`
Descrizione	Registra un nuovo utente in users.txt. Scorre l'intero file con un loop fscanf+goto per verificare l'assenza dello username. Se il file non esiste (errno==ENOENT) salta direttamente alla label do_write. Alla conferma dell'unicità apre il file in append e scrive 'user pass\n'. Corretto rispetto alla versione originale: il goto read_again dopo strcmp fallito garantisce la scansione di tutte le righe.
Parametri	**`user`** — Username da registrare, già validato lato client (no '|' o '\\'). **`pass`** — Password in chiaro da associare all'utente.
Valore di ritorno	1 se registrazione avvenuta; 0 se utente già esistente o errore I/O.

authenticate_user()

authenticate_user [server.c]	
Firma	`static int authenticate_user(const char *user, const char *pass)`
Descrizione	Verifica le credenziali confrontandole con il contenuto di users.txt. Il loop con goto auth_read_again scorre l'intero file fino alla prima corrispondenza esatta di username E password, o fino all'EOF. In caso di errore di apertura file chiama exit(EXIT_FAILURE).
Parametri	**`user`** — Username da autenticare. **`pass`** — Password da verificare.
Valore di ritorno	1 se le credenziali corrispondono a un record nel file; 0 altrimenti.

user_exists()

user_exists [server.c]	
Firma	`static int user_exists(const char *username)`
Descrizione	Verifica se uno username è presente in users.txt scorrendo l'intero file. Versione corretta: il goto user_read_again dopo un confronto fallito garantisce la scansione completa, eliminando il bug che nella versione originale causava il mancato riconoscimento di tutti gli utenti non in prima posizione nel file.
Parametri	**`username`** — Username di cui si vuole verificare l'esistenza.
Valore di ritorno	1 se lo username esiste nel file; 0 altrimenti o in caso di errore I/O.

save_message()

save_message [server.c]	
Firma	`static void save_message(Packet *pkt)`
Descrizione	Persiste un messaggio in messages.txt in formato pipe-delimited. Calcola tramite XOR tra pkt->id e un valore rand() nell'intervallo [10, 108] per ridurre le collisioni tra ID generati da client distinti. Il timestamp definitivo è quello già impostato da handle_client(). Dopo la scrittura emette un log su stdout fuori dalla sezione critica.
Parametri	**`pkt`** — Puntatore al Packet da salvare. Il timestamp è già stato impostato dal chiamante.
Valore di ritorno	void.

read_messages()

read_messages [server.c]	
Firma	`static void read_messages(const char *user, int sock)`
Descrizione	Legge messages.txt e invia al socket del client tutti i pacchetti MESSAGE_LIST il cui receiver corrisponde all'utente autenticato. Le righe malformate vengono ignorate con un warning su stderr. La trasmissione avviene interamente dentro la sezione critica di file_mutex.
Parametri	**`user`** — Username autenticato del client richiedente. **`sock`** — Descrittore del socket verso cui inviare i pacchetti MESSAGE_LIST.
Valore di ritorno	void.

delete_message()

delete_message [server.c]	
Firma	`static void delete_message(const char *user, int msg_id, int sock)`
Descrizione	Cancella un messaggio con il pattern read-filter-write. Apre messages.txt in lettura e TMP_FILE in scrittura. Copia tutte le righe tranne quella con id==msg_id && receiver==user. Chiude entrambi i file e invoca rename() per sostituire atomicamente il file originale. Le righe malformate vengono riscritte nel temporaneo con warning su stderr. Invia la risposta DELETE_MESSAGE con subject 'OK', 'NOT_FOUND' o 'ERR_IO'.
Parametri	**`user`** — Username del client: solo il destinatario può cancellare il proprio messaggio. **`msg_id`** — ID numerico del messaggio **`sock`** — Socket del client verso cui inviare la risposta.

Valore di ritorno	void. L'esito è comunicato tramite pacchetto DELETE_MESSAGE.

handle_client()

handle_client [server.c]	
Firma	`static void *handle_client(void *arg)`
Descrizione	Funzione eseguita da ogni thread worker. Libera la client_info ricevuta tramite free(). Gestisce la fase di autenticazione/registrazione iniziale: per il login, un fallimento chiude la connessione; per la registrazione, un loop while(!register_user) permette tentativi multipli senza chiudere il socket, inviando AUTH_FAIL ad ogni tentativo fallito e attendendo un nuovo AUTH_REQUEST. Dopo l'autenticazione entra nel loop di dispatching tramite switch su pkt.type.
Parametri	**`arg`** — Puntatore a client_info allocato con malloc() nel main. Liberato con free() all'inizio.
Valore di ritorno	NULL. pthread_exit(NULL) nei punti di uscita anticipata.

5. Documentazione — client.c

5.1 Funzioni di Utilità

print_packet()

print_packet [client.c]	
Firma	`static void print_packet(const Packet *pkt)`
Descrizione	Formatta e stampa su stdout le informazioni di un pacchetto MESSAGE_LIST ricevuto: ID, data/ora leggibile, mittente, oggetto e corpo. Converte il timestamp Unix tramite localtime_r() (thread-safe). Se localtime_r() fallisce stampa 'N/A' come placeholder.
Parametri	**pkt** — Puntatore costante al pacchetto da stampare.
Valore di ritorno	void.

generate_message_id()

generate_message_id [client.c]	
Firma	`static int generate_message_id()`
Descrizione	Genera un identificatore pseudo-casuale combinando time(NULL) e rand() tramite XOR, ridotto modulo 10^6. Il seed è inizializzato nel main con una combinazione a tre vie (time, getpid, indirizzo di printf) per massimizzare l'entropia. L'ID generato viene ulteriormente modificato dal server in save_message() tramite XOR casuale prima della scrittura su file.
Valore di ritorno	Intero nel range [0, 999999].

contains_forbidden()

contains_forbidden [client.c]	
Firma	`static int contains_forbidden(const char *s)`
Descrizione	Scansiona la stringa s verificando la presenza dei caratteri riservati dal protocollo: il pipe '|' (delimitatore di campo in messages.txt e users.txt) e il backslash '\\' (carattere di escape). Restituisce 1 al primo carattere proibito, senza continuare la scansione.
Parametri	**s** — Stringa null-terminated da analizzare.
Valore di ritorno	1 se la stringa contiene almeno un carattere proibito; 0 altrimenti.

fgets_validated()

`fgets_validated` [client.c]	
Firma	`static void fgets_validated(char *buf, int size, const char *field_name)`
Descrizione	Legge una riga da stdin con fgets(), rimuove il newline con strcspn(), e invoca contains_forbidden(). Se viene rilevato un carattere proibito, stampa un warning con il nome del campo e ripete la lettura tramite goto retry. In caso di errore di fgets() imposta buf[0]='\0' e ritorna senza loop infinito.
Parametri	**`buf`** — Buffer di destinazione per l'input. **`size`** — Dimensione massima del buffer passata a fgets(). **`field_name`** — Nome leggibile del campo usato nel messaggio di warning (es. 'Destinatario', 'Oggetto', 'Testo').
Valore di ritorno	void. Il risultato è in buf; buf[0]='\0' in caso di errore di lettura.

5.2 Funzione main (client)

Fase 1 — Inizializzazione e Autenticazione

Il client imposta SIG_IGN per SIGINT e SIGTSTP, inizializza rand() con seme a tre vie, crea il socket TCP e si connette a 127.0.0.1:9090. Presenta il menu login/registrazione con retry su EINTR e input non valido. Acquisisce username e password con loop che includono controllo EINTR, validazione lunghezza e contains_forbidden(). Invia AUTH_REQUEST e attende la risposta.

In caso di AUTH_FAIL durante la registrazione, il client entra nel loop di retry: stampa il messaggio di username già presente, acquisisce nuove credenziali con gli stessi controlli, invia un nuovo AUTH_REQUEST e rivaluta la risposta tramite goto retry_user_reg. Il loop continua fino all'ottenimento di AUTH_SUCCESS, senza chiudere il socket. In caso di AUTH_FAIL durante il login il client termina.

Fase 2 — Loop principale

Scelta	Operazione	Comportamento dettagliato
1	Invia messaggio	Legge destinatario, oggetto e testo con fgets_validated() dentro stdin_mutex. Avvisa se il destinatario coincide con l'utente loggato. Genera ID con generate_message_id(), imposta timestamp. Invia SEND_MESSAGE e attende CHECK_USER.
2	Leggi messaggi	Invia READ_MESSAGES, imposta SO_RCVTIMEO a 200ms, riceve tutti i MESSAGE_LIST con print_packet(). Conta i messaggi e stampa 'Nessun messaggio' se la lista è vuota. Resetta il timeout.
3	Cancella messaggio	Visualizza prima la lista messaggi (come opzione 2), poi chiede l'ID con retry su EINTR, invia DELETE_MESSAGE e stampa l'esito della risposta del server.
4	Esci	Invia END_SESSION, chiude il socket e termina con return 0.